Fase 1 — Recepción y creación de órdenes (Punto 1)

Rama: `fase-1-recepcion-ordenes` · Ejecución: 14/10/2026 · Estado: completada (pendientes en la sección 9).

Este archivo tiene dos partes:

- **Parte 1:** informe técnico de lo implementado y verificado.
- **Parte 2:** guía de estudio para la defensa oral.

Convención de rutas: `SRC` = `src/main/java/ar/edu/iua/iw3`.

Parte 1 — Informe técnico

1. Alcance y resultado

La fase implementa el Punto 1 de la consigna: un sistema externo (SAP) envía la orden con su camión, chofer, cliente y producto; el backend los sincroniza por código externo, crea la orden en estado 1 y registra el primer cambio de estado.

También aplica las decisiones aprobadas: retiro del demo (1), contenedor `iw3-gas-mysql` (10), `/error` y login (11) y `getUserLogged()` (12).

Criterio de aceptación	Resultado
Alta de orden: 201 con `location`	Cumplido
Número de orden repetido: 302	Cumplido
Datos inválidos: 400 informando todos los errores	Cumplido
Entidades sincronizadas por `externalCode` (alta, actualización, cisternado)	Cumplido
Orden en estado 1 y log de estados con un registro	Cumplido
Atomicidad: una falla al final no deja filas parciales	Cumplido
Documentación Swagger/OpenAPI	`/v3/api-docs` responde 200 con las 5 rutas; UI en `/swagger-ui.html`
Colección Postman	10 requests, 19 aserciones, 0 fallas (newman 6)

2. Decisiones de diseño

Tema	Decisión	Alternativa descartada y motivo
Entrada	`@RequestBody Order` sobre la entidad, como la cátedra con `Product`. `OrderBusiness.add` copia solo los campos permitidos	DTO de entrada: se aparta del enfoque de la cátedra; el riesgo (que el cliente fije `status` o `id`) se anula copiando campos
Estado	Enum `OrderStatus` con `AttributeConverter`: columna entera 1–4 y JSON numérico	`ORDINAL` (0–3) o `STRING`: la consigna habla de "estado 1 a 4"
Duplicado	`FoundException` → 302 (decisión 4)	409 Conflict: contradice la convención de la cátedra
Datos inválidos	Nueva `BadRequestException` → 400	`BusinessException` → 500: un 500 indica falla del servidor, no del cliente
Sincronización	Buscar por `externalCode`; actualizar o crear. El cisternado solo se reescribe si cambió	Insertar siempre: duplica camiones y repite altas y bajas de cisternas

Tema	Decisión	Alternativa descartada y motivo
Transacción	`@Transactional(rollbackFor = Exception.class)` en `add`	Sin transacción (como la cátedra): las excepciones de negocio son *checked* y Spring no revierte por ellas
Tabla de producto	`gas_products`	`products`: ya existe en bases creadas con el demo (como la de la VM) con otro esquema
Fechas	`LocalDateTime` y `hibernate.jdbc.time_zone`	`java.util.Date`: la cátedra lo usa, pero mezcla instantes y zona (hallazgo H-8)
JSON de entrada	Binding directo de Jackson	`JsonUtiles`: descartaba enteros (H-6); se retiró con el demo
Documentación	`springdoc-openapi-starter-webmvc-ui` 3.1.1	No está en el material de la cátedra: es una **explicación complementaria**

3. Archivos de la fase

3.1 Creados y modificados

Archivo	Acción	Motivo	Responsabilidad
`SRC/model/Truck.java`, `TruckTank.java`	Creados	Camión con su cisternado	Entidades
`SRC/model/Driver.java`, `Customer.java`	Creados	Chofer y cliente	Entidades
`SRC/model/Product.java`	Reemplazado	Producto de gas (tabla `gas_products`); el demo se retiró	Entidad
`SRC/model/Order.java`	Creado	Orden de carga: núcleo del proceso	Entidad
`SRC/model/OrderStatus.java`, `OrderStatusConverter.java`	Creados	Estados 1 a 4 y su mapeo a columna entera	Modelo
`SRC/model/OrderStatusLog.java`	Creado	Auditoría de cambios de estado	Entidad
`SRC/model/persistence/{Truck,Driver,Customer}Repository.java`	Creados	Búsqueda por código externo	Acceso a datos
`SRC/model/persistence/ProductRepository.java`	Reemplazado	Idem, para `Product`	Acceso a datos
`SRC/model/persistence/OrderRepository.java`, `OrderStatusLogRepository.java`	Creados	Órdenes e historial	Acceso a datos
`SRC/business/IOrderBusiness.java`, `OrderBusiness.java`	Creados	Lecturas, alta y sincronización	Negocio
`SRC/business/OrderValidator.java`	Creado	Validación de la orden recibida	Negocio
`SRC/business/exception/BadRequestException.java`	Creado	Datos inválidos → 400	Excepción
`SRC/controllers/OrderRestController.java`	Creado	Consulta de órdenes e historial	API
`SRC/integration/sap/controllers/OrderSapRestController.java`	Creado	`POST /integration/sap/orders`	API
`SRC/controllers/Constants.java`	Modificado	`URL_ORDERS`, `URL_INTEGRATION_SAP`; se quitan las del demo	Rutas
`SRC/controllers/BaseRestController.java`	Modificado	`getUserLogged()` (decisión 12)	API
`SRC/security/SecurityConfiguration.java`	Modificado	Permite `/error` (H-1, decisión 11)	Seguridad
`SRC/auth/controller/AuthRestController.java`	Modificado	Quita `produces=text/plain` (H-2, decisión 11)	Seguridad
`SRC/config/OpenApiConfig.java`	Creado	Título y esquema Bearer JWT de Swagger	Configuración
`SRC/Iw3Application.java`	Modificado	Quita los beans del demo	Arranque
`src/main/resources/application.properties`	Modificado	`hibernate.jdbc.time_zone` (H-8)	Configuración

Archivo	Acción	Motivo	Responsabilidad
`pom.xml`	Modificado	Agrega `springdoc`; quita los perfiles `cli1` y `cli2`	Build
`docker-compose.yml`, `README.md`	Modificados	Contenedor `iw3-gas-mysql` (H-4, decisión 10)	Infraestructura
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Creado	Pruebas de la API	Pruebas
`docs/fase-1-recepcion-ordenes.md`	Creado	Este informe y la guía	Documentación

3.2 Eliminados (30 archivos, decisión 1)

Carpeta (bajo `SRC` salvo indicación)	Archivos
`integration/cli1/**`	6 (controller, servicio, interfaz, deserializador, repositorio, entidad)
`integration/cli2/**`	9 (controller, servicio, interfaz, repositorio, proyección, 2 serializadores, 2 entidades)
`business`	4 (`IProductBusiness`, `ProductBusiness`, `ICategoryBusiness`, `CategoryBusiness`)
`controllers`	1 (`ProductRestController`)
`model`, `model/persistence`	2 (`Category`, `CategoryRepository`; `Product` y `ProductRepository` se reemplazaron)
`config/profile`	4 (`ScanConfig` de cli1, cli2, mysqldev y mysqlprod)
`util`	1 (`JsonUtiles`)
`src/main/resources`	2 (`application-cli1.properties`, `application-cli2.properties`)
`src/test/.../datasets`	1 (`add_postman.csv`)

Los archivos retirados siguen en el historial de git (rama `main`, commit `c3b62de`).

4. Recorrido de la petición `POST /api/v1/integration/sap/orders`

Paso	Componente	Qué hace
1	`JWTAuthorizationFilter.doFilterInternal`	Valida el JWT y deja el usuario en el `SecurityContextHolder`
2	`SecurityConfiguration` (`anyRequest().authenticated()`)	Sin token válido responde 401
3	`DispatcherServlet` + Jackson	Convierte el JSON en `Order` (con `Truck`, `Driver`, `Customer`, `Product`). JSON ilegible: 400. `Content-Type` ajeno: 415
4	`OrderSapRestController.add`	Obtiene el usuario con `getUserLogged()` y llama al negocio
5	`OrderValidator.validate`	Junta todos los errores de entrada; si hay, `BadRequestException`
6	`OrderBusiness.findByNumber`	Si el número ya existe, `FoundException`
7	`OrderBusiness.syncTruck/Driver/Customer/Product`	Busca por `externalCode`, actualiza o crea, y guarda
8	`OrderRepository.save`	`INSERT` en `orders` con estado 1 y fecha de recepción
9	`OrderBusiness.logStatus`	`INSERT` en `order_status_logs` (de `null` a 1, con el actor)
10	`OrderSapRestController.add`	Responde 201 con `location: /api/v1/orders/{orderNumber}`, sin cuerpo

Los pasos 5 a 9 ocurren en una sola transacción. Ante `FoundException` o `BadRequestException` no se escribió nada; ante cualquier otra falla se revierte todo.

5. Funciones y métodos relevantes

`OrderSapRestController.add` — `SRC/integration/sap/controllers/OrderSapRestController.java`

- *Responsabilidad:* traducir la petición HTTP del sistema externo en una llamada al negocio y el resultado en un código HTTP.
- *Recibe:* `@RequestBody Order order`. *Devuelve:* `ResponseEntity<?>`: 201 con `location`, o un `StandartResponse` de error.
- *Usa:* `getUserLogged()` (de `BaseRestController`), `IOrderBusiness.add` e `IStandartResponseBusiness.build`.
- *Errores:* `BadRequestException` → 400; `FoundException` → 302; `BusinessException` → 500.

`OrderBusiness.add` — `SRC/business/OrderBusiness.java`

- *Responsabilidad:* regla del Punto 1: validar, rechazar repetidos, sincronizar entidades, crear la orden en estado 1 y auditar.
- *Recibe:* `Order order` y `String actor`. *Devuelve:* la `Order` persistida (con `id`, estado y fecha de recepción).
- *Validaciones:* `OrderValidator.validate`; número de orden no repetido.
- *Datos:* lee `orders` y las cuatro tablas externas; escribe en `trucks`, `truck_tanks`, `drivers`, `customers`, `gas_products`, `orders` y `order_status_logs`.
- *Detalle:* crea una `Order` nueva y copia solo `orderNumber`, `plannedLoadDateTime`, `preset` y las entidades sincronizadas; `status` y `receptionDateTime` los fija el backend.
- *Errores:* `BadRequestException`, `FoundException`, `BusinessException` (cualquier otra falla, con rollback).

`OrderValidator.validate` — `SRC/business/OrderValidator.java`

- *Responsabilidad:* revisar la orden recibida sin tocar la base.
- *Recibe:* `Order`. *Devuelve:* `List<String>` con un mensaje por problema (vacía si es válida).
- *Reglas:* `orderNumber > 0`; `plannedLoadDateTime` presente; `preset > 0`; camión con código y patente, al menos una cisterna con número y capacidad `> 0` y sin números repetidos; chofer con código, nombre y apellido; cliente con código y razón social; producto con código y nombre; largos acotados a la columna.

`OrderBusiness.syncTruck` (y `syncDriver`, `syncCustomer`, `syncProduct`)

- *Responsabilidad:* aplicar el código externo como clave de integración.
- *Detalle:* `findByExternalCode(code.trim())`; si no existe crea una entidad nueva; copia los atributos del cliente (nunca su `id`) y guarda. En el camión compara el cisternado con `sameTanks` y lo reemplaza solo si cambió.

`OrderBusiness.logStatus` — registra `OrderStatusLog(order, from, to, ahora, actor, observaciones)`; si el actor llega vacío usa `"sistema"`.

`OrderStatusConverter` — `SRC/model/OrderStatusConverter.java`: `convertToDatabaseColumn` guarda el número 1–4 y `convertToEntityAttribute` lo vuelve a enum. Con `autoApply = true` rige para todo atributo `OrderStatus`.

`BaseRestController.getUserLogged` — devuelve el `User` del `SecurityContextHolder` (el que arma `JWTAuthorizationFilter`) o `null`. De ahí sale el actor del log.

6. Pruebas y verificación

Todo se ejecutó el 14/10/2026 contra MySQL real (`iw3-gas-mysql`), con el usuario `admin` del seed.

#	Verificación	Resultado
1	`./mvnw -B -q -DskipTests compile` en cada paso	`BUILD SUCCESS`
2	Arranque (`spring-boot:run`)	`Started Iw3Application in 13,1 s`; Hibernate creó `trucks`, `truck_tanks`, `drivers`, `customers`, `gas_products`, `orders`, `order_status_logs`
3	`POST` sin token	401
4	`POST` válido con `preset` entero (15000)	201, `location: /api/v1/orders/90001`
5	`GET /orders/90001`	200; `status: 1`; camión con 2 cisternas; fecha de recepción asignada

#	Verificación	Resultado
6	`GET /orders/90001/status-log`	1 registro: `fromStatus: null`, `toStatus: 1`, `actor: admin`
7	Mismo `POST` otra vez	302 con `StandartResponse`
8	Orden con campos inválidos	400 con los 10 errores en un solo mensaje
9	JSON malformado, fecha inválida, `Content-Type` erróneo	400, 400, 415
10	Orden inexistente, número no numérico, ruta inexistente	404, 400, 404
11	Login con contraseña incorrecta	401 **con** `StandartResponse`
12	Sincronización: mismo camión con otra patente y mismas cisternas	201; 1 camión, patente actualizada, cisternas con los mismos ids
13	Sincronización: mismo camión con 3 cisternas	201; cisternas reemplazadas (3 filas nuevas)
14	Sincronización: cliente con razón social distinta	Una sola fila de cliente, con la razón social nueva
15	Atomicidad: descripción de producto de 300 caracteres (columna de 255)	500 y **0 filas** de camión, chofer, cliente, producto y orden
16	`GET /v3/api-docs` y `/swagger-ui.html`	200 con las 5 rutas; 302 a `/swagger-ui/index.html`
17	`newman run` de la colección	10 requests, 19 aserciones, 0 fallas

La colección ejecutada con newman es idéntica byte a byte a `docs/postman/IW3-Gas-Terminal.postman_collection.json`.

Fechas: medición del hallazgo H-8

Caso	JVM	URL	`hibernate.jdbc.time_zone`	En la base	En el JSON
Corrida inicial	Argentina	dev	no	17:49:33	14:49:33
B	UTC	dev	no	17:51:57	14:51:57
C	UTC	prod	no	17:52:20	14:52:20
D	Argentina	dev	sí	14:52:42	14:52:42
E	UTC	dev	sí	14:53:04	14:53:04
F	UTC	prod	sí	14:53:26	14:53:26

Sin la propiedad, MySQL recibía la hora en UTC (3 h adelantada) aunque el JSON se viera bien. Con la propiedad, la base guarda la hora de Argentina en los seis escenarios.

7. Cambios de comportamiento respecto de la Fase 0

Petición	Fase 0	Fase 1
JSON malformado con token válido	401 sin cuerpo	400
`Content-Type` erróneo	401 sin cuerpo	415
Ruta inexistente	401 sin cuerpo	404
Login con contraseña incorrecta	401 sin cuerpo	401 con `StandartResponse`
`/api/v1/products` y `/categories`	200	404 (retirados)
Fechas en la base	UTC	Hora de Argentina

8. Hallazgos y limitaciones de la fase

ID	Hallazgo	Estado
N-1	`OrderValidator.validate` no invocaba la validación de chofer, cliente y producto: al partir el commit se perdieron las tres llamadas. Lo detectó la prueba del 400	Corregido (`fix(business)`)
N-2	Dos `POST` simultáneos con el mismo número de orden: uno viola la restricción única y responde 500 en lugar de 302	Pendiente: se acepta para el parcial
N-3	`GET /api/v1/orders` devuelve todas las órdenes, sin paginación	Pendiente: a revisar con el frontend
N-4	Las tablas del demo (`products`, `categories`, `cli1_products`, `cli2_*`) siguen en las bases ya creadas; `ddl-auto=update` no borra	Inofensivo: el TP usa `gas_products`
N-5	Las respuestas de error pesan unos 15 KB porque `dev.info.enabled=true` incluye la traza (H-5)	Pendiente para la etapa de seguridad
N-6	La ejecución con newman usó `npx newman@6` (descarga a la caché de npm; no se agrega al proyecto)	Informativo

Sin cambios: H-5 (secretos, `/demo/**`, CORS abierto) y H-9 (desalineación del `pom.xml`, decisión 6).

9. Pendientes y criterios de la fase

Pendiente	Cuándo
Código HTTP para "estado incorrecto" (propuesta: 409 o 400)	Fase 2
Roles por endpoint (SAP, TMS, sistema de carga)	Examen final
Revisión del equipo de la Fase 1 (la rama ya se integró a `main`: PR #2, `a330219`, 14/10/2026, sin revisiones registradas)	Pendiente
Desplegar en la VM: imagen nueva desde `main` y variables `SPRING_DATASOURCE_*`	Por definir

Criterio	Estado
Funcionalidad implementada	Cumplido
Metodología de la cátedra respetada (capas, `I*Business`, excepciones, `StandartResponse`, `Constants`)	Cumplido; desvíos justificados en la sección 2
Verificación con pruebas ejecutadas	Cumplido (sección 6)
Archivos documentados	Cumplido (sección 3)
Funciones explicadas	Cumplido (sección 5)
Recorrido de los datos	Cumplido (secciones 4 y E)
Guía de estudio A–J con preguntas y respuestas	Cumplido
Limitaciones registradas	Cumplido (sección 8)

Parte 2 — Guía de estudio de la Fase 1

A. Objetivo de la fase

El Punto 1 es la **puerta de entrada** del circuito: sin una orden creada no hay tara, ni carga, ni cierre, ni conciliación. SAP envía los datos base y el backend debe:

1. Aceptar solo datos válidos y rechazar un número de orden repetido.

2. Sincronizar camión, chofer, cliente y producto usando el **código externo** como clave de integración (la consigna lo pide como `string` para no depender de ids internos).
3. Crear la orden en estado 1 y dejar registrado quién y cuándo la recibió.

Además prepara lo que usan todas las fases siguientes: el modelo, el log de estados y la documentación Swagger y Postman.

B. Conceptos teóricos involucrados

Concepto	Clase	Dónde está en el TP
Entidad JPA, `@ManyToOne`, `@OneToMany`	2 y 6	`SRC/model/Order.java`, `SRC/model/Truck.java`
Interfaz + implementación + `@Autowired`	2	`IOrderBusiness` y `OrderBusiness`
Excepciones de dominio traducidas a HTTP (404, 302, 500)	3	`OrderSapRestController.add`
Código externo único como clave de integración (`codCli1`)	6	`externalCode` de `Truck`, `Driver`, `Customer`, `Product`
Paquete de integración por sistema externo	6	`SRC/integration/sap/controllers`
Recepción de JSON de un sistema externo	7	`@RequestBody Order` (la cátedra usó un deserializador propio)
Usuario autenticado en el controller	8 y 9	`BaseRestController.getUserLogged`

Explicación complementaria (no vista en clase):

- *`@Transactional` y `rollbackFor`:* una transacción agrupa varias escrituras. Spring revierte por defecto solo ante excepciones *unchecked*; las de negocio son *checked*, por eso se indica `rollbackFor = Exception.class`.
- *`AttributeConverter`:* traduce un tipo Java (el enum) a un valor de columna (el entero).
- *OpenAPI / springdoc:* genera la especificación a partir de los controllers y anotaciones.
- *`LocalDateTime` y zona horaria:* no lleva zona; el driver la aplica al hablar con MySQL.

C. Arquitectura y organización del código

```
SAP ── HTTP + JWT ──▶ OrderSapRestController      integration/sap/controllers
                          │  Order (JSON → entidad)
                          ▼
                      OrderBusiness (IOrderBusiness)         business
                       ├─ OrderValidator            reglas de entrada
                       ├─ sync* (Truck, Driver, Customer, Product)
                       └─ logStatus
                          ▼
                      Repositorios Spring Data               model/persistence
                          ▼
                      MySQL: orders, order_status_logs, trucks, truck_tanks,
                             drivers, customers, gas_products
```

Pieza	Responsabilidad	Por qué está ahí
`OrderSapRestController`	HTTP ↔ negocio: código de estado y `location`	Es la única capa que conoce HTTP
`OrderValidator`	Reglas de la entrada	Separa "es válida" de "se guarda"; se prueba sin base
`OrderBusiness`	Sincroniza, crea y audita en una transacción	Las reglas del proceso van en negocio, no en el controller
`*Repository`	Acceso a datos por código externo	Spring Data genera las consultas
`OrderStatus` + converter	Estados 1 a 4	Un solo lugar define qué estados existen
`OrderStatusLog`	Auditoría	La consigna pide registrar cada cambio de estado

D. Explicación del código paso a paso

D.1 El alta — `SRC/business/OrderBusiness.java`

```
@Override
@Transactional(rollbackFor = Exception.class)
public Order add(Order order, String actor) throws FoundException, BadRequestException, BusinessException {
    List<String> errors = OrderValidator.validate(order);
    if (!errors.isEmpty()) {
        throw BadRequestException.builder().message(String.join("; ", errors)).build();
    }
    if (findByNumber(order.getOrderNumber()).isPresent()) {
        throw FoundException.builder().message("Se encontró la Orden numero=" + order.getOrderNumber()).build();
    }
    try {
        Order o = new Order();
        o.setOrderNumber(order.getOrderNumber());   // ... plannedLoadDateTime, preset
        o.setTruck(syncTruck(order.getTruck()));    // ... driver, customer, product
        o.setStatus(OrderStatus.PENDING_INITIAL_WEIGHING);
        o.setReceptionDateTime(LocalDateTime.now());
        o = orderDAO.save(o);
        logStatus(o, null, o.getStatus(), actor, "Orden recibida desde el sistema externo");
        return o;
    } catch (Exception e) { /* log + BusinessException */ }
}
```

Elemento	Qué hace	Si se elimina o cambia
`@Transactional(rollbackFor = Exception.class)`	Todo el método es una transacción y se revierte ante cualquier excepción	Una falla al sincronizar el producto dejaría creados camión, chofer y cliente sin orden
`OrderValidator.validate` primero	Rechaza antes de tocar la base y junta todos los errores	El cliente corregiría un error por intento, o llegaría a la base con datos inválidos
Chequeo del número **fuera** del `try`	`FoundException` sale tal cual	Dentro del `try`, el `catch (Exception)` la convertiría en `BusinessException` (500 en vez de 302)
`new Order()` y copia de campos	El cliente no controla `id`, `status` ni `receptionDateTime`	Con `save(order)` un cliente podría enviar `status: 4` o un `id` ajeno
`sync*` antes de `save`	La orden referencia filas que ya existen	`save` fallaría por claves foráneas inexistentes
`logStatus` en la misma transacción	El registro de auditoría existe si y solo si existe la orden	Una orden sin historial, o un historial de una orden inexistente

D.2 La sincronización — `OrderBusiness.syncTruck`

```
String code = in.getExternalCode().trim();
Truck t = truckDAO.findByExternalCode(code).orElseGet(Truck::new);   // existe o se crea
t.setExternalCode(code);
t.setPlate(in.getPlate().trim());
List<TruckTank> incoming = /* copia de las cisternas recibidas, sin sus ids, ordenadas */;
if (!sameTanks(t.getTanks(), incoming)) {       // solo si cambió el cisternado
    t.getTanks().clear();                       // orphanRemoval borra las anteriores
    t.getTanks().addAll(incoming);
}
return truckDAO.save(t);
```

- `findByExternalCode(...).orElseGet(Truck::new)` es el corazón de la sincronización: el **código externo** decide si se actualiza o se crea; el `id` que mande SAP se ignora.
- `t.getTanks().clear()` más `addAll` (y no `setTanks(nuevaLista)`) porque la colección tiene `orphanRemoval = true`: reemplazar la referencia haría fallar a Hibernate.
- `sameTanks` evita borrar y reinsertar cisternas idénticas en cada orden del mismo camión (verificado: los ids de las cisternas no cambian).

D.3 El estado — `OrderStatus` y `OrderStatusConverter`

- `OrderStatus` define los cuatro estados con su número; `@JsonValue` en `getCode()` hace que el JSON publique `1` y no `"PENDING_INITIAL_WEIGHING"`.
- `OrderStatusConverter` (`autoApply = true`) guarda ese número en la columna. Sin él, Hibernate usaría el ordinal 0–3 y la base mostraría "0" para el estado 1 de la consigna.

E. Trazabilidad de los datos

Pregunta	Alta de la orden (`POST /integration/sap/orders`)	Consulta (`GET /orders/{n}`)
¿De dónde vienen?	Cuerpo JSON que envía SAP	Parámetro de la ruta
¿Cómo llegan?	`@RequestBody Order`: Jackson arma `Order` con sus cuatro entidades anidadas y la fecha ISO	`@PathVariable Integer orderNumber`
¿Dónde se validan?	`OrderValidator.validate` y número no repetido (`findByNumber`)	`OrderBusiness.load` (si no existe, `NotFoundException`)
¿Dónde se transforman?	`OrderBusiness.add`: copia de campos permitidos, `trim`, estado 1, fecha de recepción; `sync*` por código externo	No se transforman: la entidad se serializa tal cual
¿Dónde se almacenan?	`trucks`, `truck_tanks`, `drivers`, `customers`, `gas_products`, `orders`, `order_status_logs`	—
¿Cómo se recuperan?	—	`OrderRepository.findByOrderNumber`
¿Quién los usa después?	Fase 2 (tara) y siguientes: cargan la orden por número y comprueban su estado	El frontend del examen final
¿Qué recibe el cliente?	201 y `location`; o 302, 400 o 500 con `StandartResponse`	La orden en JSON, con `status` numérico

El actor del log sale del token: `JWTAuthorizationFilter` → `SecurityContextHolder` → `getUserLogged()` → `OrderBusiness.logStatus` → columna `actor` de `order_status_logs`.

F. Ejemplo práctico de ejecución

Los valores son ficticios; el usuario `admin` es el de prueba local.

1. Petición

```
POST /api/v1/integration/sap/orders
Authorization: Bearer <JWT de admin>
Content-Type: application/json

{ "orderNumber": 90001, "plannedLoadDateTime": "2026-10-20T08:00:00", "preset": 15000,
  "truck":    { "externalCode": "TRK-001", "plate": "AB123CD",
               "tanks": [ {"number": 1, "capacityLiters": 10000}, {"number": 2, "capacityLiters": 5000} ] },
  "driver":   { "externalCode": "DRV-001", "name": "Juan", "lastName": "Perez" },
  "customer": { "externalCode": "CUS-001", "companyName": "Gas del Sur SA" },
  "product":  { "externalCode": "PRD-GLP", "name": "GLP" } }
```

2. Validaciones que atraviesa: número `> 0`, fecha presente, `preset > 0`, camión con código, patente y cisternas válidas, chofer, cliente y producto completos, y número no usado.

3. Cambios en la base (primera vez):

Tabla	Filas nuevas
`trucks`	1 (`TRK-001`)
`truck_tanks`	2 (10000 y 5000 litros)

Tabla	Filas nuevas
`drivers`, `customers`, `gas_products`	1 en cada una
`orders`	1 (`order_number = 90001`, `status = 1`, `reception_date_time = ahora`)
`order_status_logs`	1 (`from_status = NULL`, `to_status = 1`, `actor = admin`)

4. Respuesta: `201 Created` con `location: /api/v1/orders/90001`.

5. Repetir la petición: 302 con `{"message": "Se encontró la Orden numero=90001", "code": 302, ...}` y sin escrituras. **Con `preset: -1`:** 400 con el mensaje de todos los campos inválidos.

G. Preguntas posibles del profesor

1. ¿Qué recorrido sigue una orden desde que SAP la envía hasta que queda guardada?
2. ¿Por qué la sincronización y las reglas están en `OrderBusiness` y no en el controller?
3. ¿Qué significa sincronizar por código externo? ¿Qué pasa si el camión ya existe con otra patente?
4. ¿Cómo se evita que SAP fije el estado, el id o la fecha de recepción de la orden?
5. ¿Qué pasa si falla algo a mitad de la sincronización?
6. ¿Para qué sirve `rollbackFor = Exception.class` si ya hay `@Transactional`?
7. ¿Por qué un número repetido responde 302 y un dato inválido 400?
8. ¿Dónde se valida y qué diferencia hay entre validar y persistir?
9. ¿Cómo se guarda el estado de la orden y por qué hace falta un conversor?
10. ¿Cómo se registra el cambio de estado y quién figura como actor?
11. ¿Por qué el producto está en la tabla `gas_products` y las órdenes en `orders`?
12. ¿Qué problema de fechas encontraron y cómo lo resolvieron?
13. ¿Cómo demostrarías que esta funcionalidad funciona?
14. ¿Qué se retiró del código de la cátedra y por qué?

H. Respuestas modelo

1. SAP hace `POST /api/v1/integration/sap/orders` con un JWT. `JWTAuthorizationFilter` autentica, Jackson arma el `Order` con sus cuatro entidades y `OrderSapRestController.add` llama a `OrderBusiness.add`. Ahí `OrderValidator` revisa los datos, `findByNumber` descarta repetidos, los `sync*` buscan o crean camión, chofer, cliente y producto por código externo, se guarda la orden en estado 1 y `logStatus` registra el primer cambio. El controller responde 201 con el header `location`.

2. Porque son reglas del proceso, no de HTTP: validar, no repetir, sincronizar y auditar. El controller solo traduce a HTTP (códigos de estado y `location`). Con esa separación la regla se puede reutilizar y probar sin pasar por HTTP, y es el mismo criterio que usa la cátedra con `ProductBusiness`.

3. Que el sistema externo identifica cada entidad con su propio código y el backend lo usa como clave de integración, no el `id` interno. `findByExternalCode` decide: si existe se actualizan sus datos (por ejemplo la patente) y si no se crea. Lo verifiqué: una segunda orden del mismo camión con otra patente dejó una sola fila de camión con la patente nueva.

4. `add` no guarda el objeto que llega: crea un `Order` nuevo y copia solo `orderNumber`, `plannedLoadDateTime`, `preset` y las entidades ya sincronizadas. `status` y `receptionDateTime` los asigna el backend, y los `id` del cliente se ignoran. En Swagger esos campos figuran como solo lectura.

5. Se revierte todo. El método es una única transacción, así que si, por ejemplo, el producto falla al guardarse, también se deshacen el camión, el chofer y el cliente que ya se habían insertado. Lo probé con una descripción de 300 caracteres en una columna de 255: respuesta 500 y cero filas nuevas.

6. Porque por defecto Spring revierte solo ante excepciones *unchecked*, y las de negocio de la cátedra (`BusinessException`, `FoundException`) son *checked*. Sin `rollbackFor`, el método terminaría con una excepción y la transacción se confirmaría igual, dejando filas sueltas.

7. El 302 es la convención de la cátedra: `FoundException` se traduce a `HttpStatus.FOUND` en todos los controllers; lo dejé así por coherencia (decisión 4), aunque 409 sería más preciso. El 400 es una extensión mía: `BadRequestException`, porque un dato inválido es un error del cliente y no una falla del servidor como implicaría el 500.

8. La validación (`OrderValidator`) mira el contenido y responde "es válido o no", sin tocar la base; la persistencia guarda. Se valida primero para no escribir nada ante datos inválidos, y además junta todos los errores en un solo mensaje. El único control que necesita la base (número repetido) está después, en `OrderBusiness.add`.

9. `OrderStatus` es un enum con el número de cada estado. `OrderStatusConverter` (un `AttributeConverter` con `autoApply`) guarda ese número en la columna, porque sin él Hibernate guardaría el ordinal 0–3 y la consigna habla de estados 1 a 4. En el JSON, `@JsonValue` publica el número.

10. En `OrderBusiness.add`, dentro de la misma transacción, se inserta un `OrderStatusLog` con estado anterior `null`, nuevo 1, fecha y hora, observación y actor. El actor sale del token: el filtro deja el `User` en el `SecurityContextHolder` y el controller lo obtiene con `getUserLogged()`. En las pruebas figura `admin`.

11. `orders` porque `ORDER` es palabra reservada de SQL. `gas_products` porque en las bases creadas con el demo de la cátedra ya existe una tabla `products` con otro esquema (incluida la de la VM), y `ddl-auto=update` no la reemplazaría: el despliegue fallaría.

12. Sin configuración extra, MySQL recibía la hora en UTC: en la base figuraba `17:49` cuando eran las `14:49`, aunque el JSON se viera bien porque el ida y vuelta compensaba. Lo medí con la JVM en Argentina y en UTC y con la URL de desarrollo y la de producción, y lo resolví con `spring.jpa.properties.hibernate.jdbc.time_zone=America/Argentina/Buenos_Aires`: en los seis casos la base guarda la hora local.

13. Con tres evidencias: la colección Postman (10 requests y 19 aserciones, 0 fallas con newman), la batería de peticiones con sus códigos (201, 302, 400, 404, 415, 401) y la consulta directa a la base para ver las filas, el estado numérico y la hora. Además probé la sincronización con camiones repetidos y la atomicidad.

14. El demo de productos y categorías, los paquetes `cli1` y `cli2`, los `ScanConfig` y `JsonUtiles` (decisión 1). Eran un ejemplo didáctico que ya no aportaba al dominio y generaba confusión (incluso un nombre en común, `Product`). Quedan en el historial de git. Se conservó todo lo que sirve de infraestructura: capas, excepciones, `StandartResponse`, `Constants` y la seguridad JWT.

I. Errores frecuentes y puntos difíciles

Confusión o error	Lo correcto	Dónde se ve
Guardar directamente el `Order` que llega	Copiar solo los campos permitidos: si no, el cliente fija `status` o `id`	`OrderBusiness.add`
Creer que `@Transactional` revierte siempre	Solo ante excepciones *unchecked*, salvo `rollbackFor`	`OrderBusiness.add`
Lanzar `FoundException` dentro del `try` con `catch (Exception)`	Se convierte en `BusinessException` (500 en vez de 302)	Orden de las comprobaciones en `add`
Reemplazar la lista de cisternas con `setTanks(...)`	Con `orphanRemoval = true` hay que mutar la misma colección (`clear` + `addAll`)	`syncTruck`
Pensar que el JSON de entrada pierde enteros como con `JsonUtiles`	El binding directo de Jackson acepta `15000` como `double`	Prueba 4
Creer que 302 es un error de Spring	Es la convención de la cátedra para `FoundException`	`OrderSapRestController.add`

Confusión o error	Lo correcto	Dónde se ve
Mirar solo el JSON para validar la hora	Hay que mirar también el valor crudo de la base (el ida y vuelta lo enmascaraba)	Sección 6, H-8
Suponer que `ddl-auto=update` limpia el demo	No borra tablas ni columnas: por eso `gas_products`	N-4
Olvidar que el validador llama a cada sub-validación	Un método existente pero sin invocar no valida nada	Hallazgo N-1

Qué conviene estudiar con más profundidad:

- Cómo Spring crea el proxy de `@Transactional` y qué pasa con las llamadas internas (`this.metodo()` no pasa por el proxy).
- Diferencia entre `CascadeType.ALL`, `orphanRemoval` y `fetch = EAGER` en `Truck.tanks`.
- Por qué `Order` mapea a `orders` y qué otros nombres reservados hay que evitar.

J. Lista de comprobación para rendir

Puedo defender esta fase si, sin mirar el material, logro:

- [ ] Explicar el recorrido de `POST /integration/sap/orders` con archivo y método de cada paso.
- [ ] Explicar qué es el código externo y qué decide `findByExternalCode(...).orElseGet(...)`.
- [ ] Explicar por qué `add` copia campos en lugar de guardar lo recibido.
- [ ] Explicar la transacción: qué se revierte, por qué `rollbackFor` y cómo lo probé.
- [ ] Justificar 201, 302, 400, 404 y 500, y decir cuál es convención de la cátedra y cuál no.
- [ ] Explicar cómo se guarda el estado (enum, conversor, número 1–4).
- [ ] Explicar de dónde sale el actor del log de estados.
- [ ] Explicar el problema de la hora en la base y cómo se resolvió.
- [ ] Ejecutar la colección Postman y leer el resultado.
- [ ] Explicar qué se retiró del demo y por qué `Product` usa `gas_products`.